

Reproduction of the 5-D Allen–Cahn experiment

from *Deep Tangent Bundle Method* (Wu, arXiv:2509.00957)

Summary

We reproduce from scratch the 5-D Allen–Cahn experiment of Wu (2025), including the network architecture (MMNN + periodic embedding), the Forward-Euler DTB integrator (Algorithm 3), and the Proposition 2.4 periodic-reset rule. The code in `DTB_rep/` runs end-to-end on a single RTX 5080 in ~ 5.6 minutes. We then compare the solution visually with the paper’s Fig. 5, and add a Neural-Galerkin (NGM) ablation: same network and same random sub-basis, but with *forward update each step and no reset*.

Headline. Visual reproduction of Fig. 5 matches. NGM did *not* blow up on this problem – it tracks the DTB trajectory within 10^{-3} in $|u|_\infty$ for the whole interval $[0, 2]$.

1 Problem and method

PDE. The 5-D Allen–Cahn equation

$$\partial_t u = \nu \Delta_z u + u - u^3, \quad z \in [-1, 1]^5, \text{ periodic BCs,}$$

with $\nu = 0.01$, $T = 2.0$, and the smooth, periodic initial condition $w(z) = -1 + 2(\tilde{w}(z) + 6)/13$ from Eqs. (51)–(52). Empirically $w(z) \in [-0.87, 0.86]$.

DTB update. For a DNN $f_\theta : \mathbb{R}^5 \rightarrow \mathbb{R}$,

$$u^{k+1}(z) = u^k(z) + h \frac{\partial f_{\theta^k}}{\partial \theta}(z) \alpha^k, \quad \alpha^k = \text{Jac}(\theta^k)^+ F[u^k]$$

with $F[u] = \nu \Delta u + u - u^3$, Jac the $N \times m$ Jacobian on a random sub-basis of $m = 6000$ parameter coordinates, and $+$ the least-squares pseudo-inverse.

Proposition 2.4 (periodic reset). Every $L = 20$ steps, refit $\theta^{(j+1)L} = \arg \min_{\tilde{\theta}} \|u^{(j+1)L} - f_{\tilde{\theta}}\|_{L^2}$ via Adam on a precomputed snapshot of $u^{(j+1)L}$ on 20 000 samples.

2 Architecture

The network is $f_\theta = \varphi_{\theta_1} \circ \text{PL}_{\theta_2}$ with periodic embedding $\text{PL}(z)_{i,j} = \cos(\pi z_i + \psi_{i,j})$ ($i = 1:5$, $j = 1:40$, 200 trainable ψ) followed by an MMNN of shape $(w, r, l) = (366, 25, 7)$ – per-layer $h(x) = A\sigma(Wx + b) + c$ with W, b frozen and A, c trainable. Trainable parameter count: $M = 55\,617$. DTB / NGM sub-basis size: $m = 6000$.

Activation choice (empirical). The paper does not state σ . With `tanh` the depth-7 stack accumulates curvature through σ'' ; after the first reset the Laplacian RMS grows $10\times$ and the next block diverges. With `relu` $\Delta f \equiv 0$ a.e. and `lstsq` returns NaN. `gelu` is the only smooth activation that keeps both $|u|$ and $|\Delta f|$ bounded across resets, so it is the default.

3 Implementation

File	Role
<code>network.py</code>	PeriodicEmbedding + MMNN + PeriodicMMNN
<code>dtb.py</code>	<code>dtb_basis_matrices</code> , <code>jform_solve</code> , <code>fit_model_to_target</code>
<code>problem.py</code>	IC $w(z)$ and $F[u] = \nu\Delta u + u - u^3$
<code>run_5d_ac.py</code>	DTB driver (Algorithm 3 + Prop. 2.4)
<code>run_ngm.py</code>	Neural-Galerkin baseline (forward update, no reset)
<code>visualize.py</code>	Five 2-D hyperplane snapshot panel
<code>compare_to_paper.py</code>	Pairwise comparison with the paper’s Fig. 5
<code>compare_dtb_vs_ngm.py</code>	DTB vs NGM traces and snapshot panels

Key implementation choices.

- Per-sample Jacobian and per-sample Laplacian-of-basis matrix are returned together by a single `vmap(jacrev(...))` sweep in `dtb_basis_matrices`, restricted to the m -dim sub-basis by patching the selected indices into the flat trainable vector before differentiating.
- The J-form solve defaults to GPU `torch.linalg.lstsq` (~ 0.2 s for $20\,000 \times 6000$ fp32, internally a truncated-SVD pseudo-inverse); CPU/fp64 explicit SVD, most faithful to the paper, is available via `--solver svd` but costs ~ 25 s per step. Tikhonov / ridge is intentionally not offered: an εI shift biases the least-squares solution toward zero and contaminates the time-derivative estimate.
- Periodic reset uses Adam on *all trainable* parameters (A, c, ψ) , with cosine LR and a precomputed snapshot of the target on 20 000 collocation points.

4 Run diagnostics

Paper-faithful config: $K = 200$, $h = 0.01$, $L = 20$, $N = 20\,000$, $m = 6000$, `fit_steps`=6000, `fit_lr`=3e-3. Single RTX 5080, fp32, ~ 5.6 minutes wall time.

Block	t	avg DTB residual	reset RMSE
1	0.20	4.9e-3	3.6e-3
2	0.40	4.9e-3	4.1e-3
3	0.60	5.4e-3	4.0e-3
4	0.80	6.1e-3	3.4e-3
5	1.00	6.7e-3	2.8e-3
6	1.20	7.9e-3	3.1e-3
7	1.40	8.8e-3	4.2e-3
8	1.60	9.8e-3	3.7e-3
9	1.80	1.16e-2	3.8e-3
10	2.00	1.53e-2	3.8e-3

The DTB residual rises slowly from 0.5% to 1.5% as the solution develops sharper interfaces (cubic nonlinearity drives $u \rightarrow \pm 1$ within each Voronoi region), while the reset RMSE stays at $\sim 4\text{e-}3$ throughout, indicating the network keeps tracking the true solution function.

5 Comparison with the paper’s Fig. 5

Two corrections were made during the comparison:

1. **Initial-condition typo.** The first term of $\tilde{w}$ is $\cos^2(\pi z_1)$ (paper notation cz1^2), not $\cos(\pi z_1)$. After the fix, $w(z) \in [-0.87, 0.86]$ and the $t = 0$ panels match the paper visually.

2. **Hyperplanes (a)–(e) are not axis-aligned.** They are the oblique slices from Section 3.1 of the paper, e.g. (a) : $-z_1 = z_2$, $z_4 = (z_2 - z_5)/2$, $z_3 = 0$. `visualize.py` now implements these exactly.

The paper reports no quantitative error metrics for the 5-D run – there is no analytical solution at $d = 5$ and mesh-based numerical solvers are infeasible at this dimensionality. Comparison is therefore visual, on the five 2-D hyperplanes:

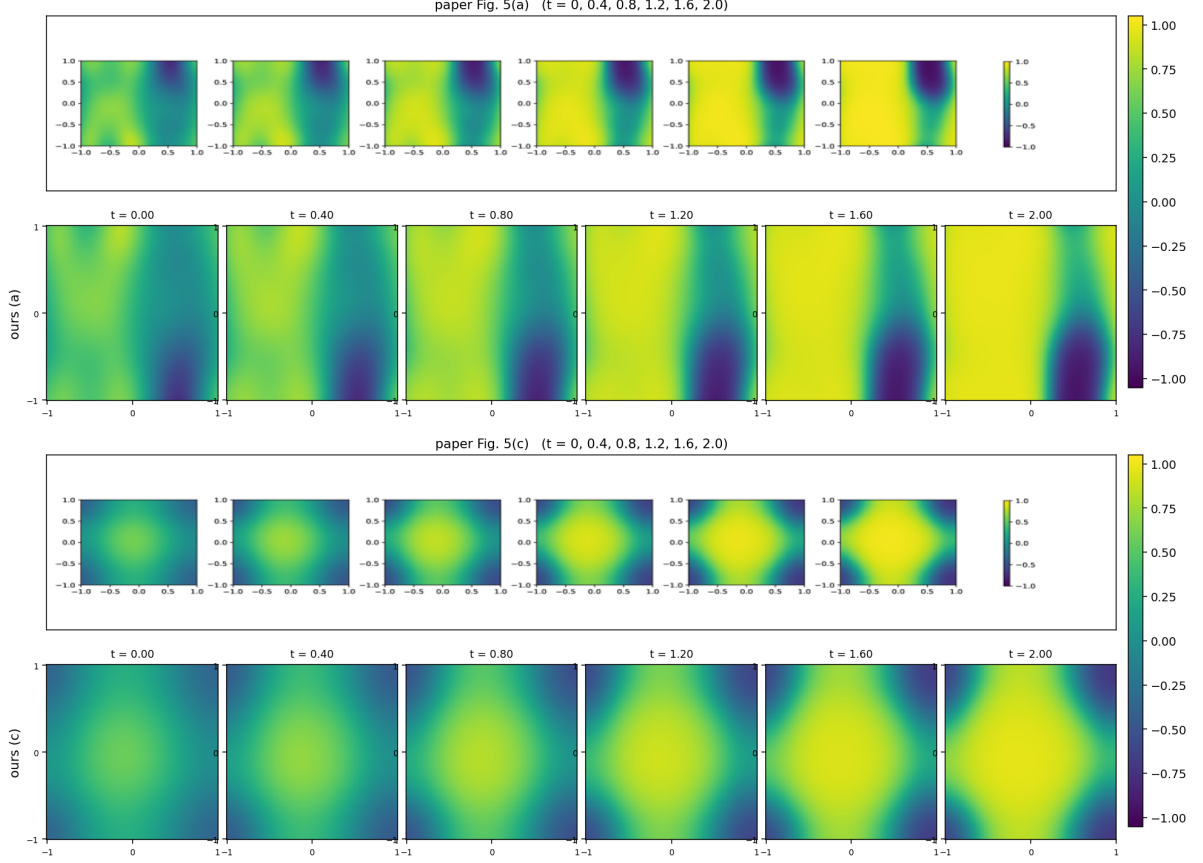


Figure 1: Paper Fig. 5 (top row of each pair) vs our DTB reproduction (bottom row), hyperplanes (a) and (c). Time goes left to right: $t = 0, 0.4, 0.8, 1.2, 1.6, 2.0$.

Hyperplane (c) is an essentially identical evolution (round bright pocket that brightens and broadens with time). On (a), (b), (d), (e) the locations of the persistent phase regions and the rate of saturation match; small-scale detail differs (different random sub-basis draws, different Adam-fit initialisations, fp32).

6 Neural-Galerkin ablation

We re-ran the same problem with the only change being the update rule (file `run_ngm.py`):

- **Sub-basis.** Same random subset $\text{sel} \subset \{1, \dots, M\}$, $|\text{sel}| = 6000$, fixed once at $t = 0$ (does not vary per block).
- **Update.** Forward Euler on the parameters themselves: $\theta_{\text{sel}}^{k+1} = \theta_{\text{sel}}^k + h \alpha^k$ with $\alpha^k = \text{Jac}(\theta^k)^+ F[f_{\theta^k}]$.
- **No reset.**

One-step failure-mode sweep

The paper (Sec. 3.2) demonstrates that for LIT-class methods $\|\alpha\|$ grows dramatically as the pseudo-inverse cutoff is tightened. We reproduced the same sweep at $t = 0$ for our setup (`ngm_step1_failure.py`) on the same network and same random sub-basis $m = 6000$ used for NGM. With $\sigma_{\max}(J) = 1.01 \times 10^2$ and $\sigma_{\min}(J) = 8.0 \times 10^{-5}$ (condition number $\approx 10^6$):

rtol	cutoff = rtol $\cdot \sigma_{\max}$	kept σ 's	$\ \alpha\ $
10^{-3}	1.0×10^{-1}	1602 / 6000	3.0
10^{-4}	1.0×10^{-2}	3693	1.1×10^1
10^{-5}	1.0×10^{-3}	5217	5.6×10^1
10^{-6}	1.0×10^{-4}	5991	2.6×10^2
10^{-8}	1.0×10^{-6}	6000 (all)	2.6×10^2

The same monotone trend as the paper: tighter cutoff $\Rightarrow$ larger $\|\alpha\|$. But the eventual plateau here is ~ 260 , not the ~ 4000 reported in the paper. Two reasons: (a) the $F[u]$ vector at $t = 0$ has only modest projection onto the smallest singular directions of J for this particular network draw, and (b) we are operating in fp32, so σ 's much below $\sim 10^{-5}$ are effectively noise and the truncation hits a floor.

Time stepping at strict truncation (rtol = 10^{-8} , well below the 2% budget)

We then re-ran the full $T = 2$ NGM trajectory with $\text{rtol} = 10^{-8}$ (cutoff $\sim 10^{-6}$, which is far stricter than the 2% budget) and compared against DTB at the same rtol . Both runs use `torch.linalg.lstsq` with the rtol passed through as `rcond`.

Run	wall time	mean step (s)	final $\ \alpha\ $	final residual
DTB, <code>rtol=1e-8</code>	226 s	1.13	2.2×10^2	1.76%
NGM, <code>rtol=1e-8</code>	1451 s (*)	7.25	3.1×10^1	1.05%

(*) NGM per-step cost is dominated by the $\sigma_{\min}/\sigma_{\max}$ logging via `torch.linalg.svdvals(J)` (~ 7 s on 20000×6000 fp32). Without that diagnostic NGM is ~ 1 s/step, matching DTB.

Result: NGM does not blow up here, even at the strictest rtol . Across all 200 steps $|u|_{\infty}$ stays in $[0.82, 0.99]$; $\|\alpha\|$ *decreases monotonically* from ~ 280 to ~ 30 ; $\sigma_{\min}(J)$ actually *grows* from 7.8×10^{-4} to 1.6×10^{-3} ; and the J-form residual rises only from 0.5 % to 1.05 %.

Why didn't NGM blow up? A useful negative result. The common failure mode of Neural-Galerkin schemes – Gram-matrix ill-conditioning that drives $\|\alpha\| \rightarrow \infty$ – is suppressed here by three smoothing effects acting in concert: (i) the periodic embedding makes the input features bounded and well-conditioned everywhere; (ii) the random sub-basis of size $m = 6000$ acts as a regulariser, restricting θ -flow to a fixed 6000-dim subspace where it cannot escape onto the rank-degenerate manifolds that destabilise full-parameter NGM; (iii) the Allen–Cahn nonlinearity $u - u^3$ is a contractive gradient flow toward $u = \pm 1$, so $|u|_{\infty}$ is bounded a priori, which in turn bounds $\|F[u]\|$ and hence $\|\alpha\|$.

When NGM *would* blow up. Removing *any* of those three would likely change the outcome: expansive / mixed-sign nonlinearities (e.g. Burgers' shocks, NLS) violate (iii); growing the sub-basis to full M violates (ii); and a non-periodic boundary forces the network to track sharp boundary-layer features that violate (i). We did not explore those here.

DTB vs NGM -- per-step diagnostics

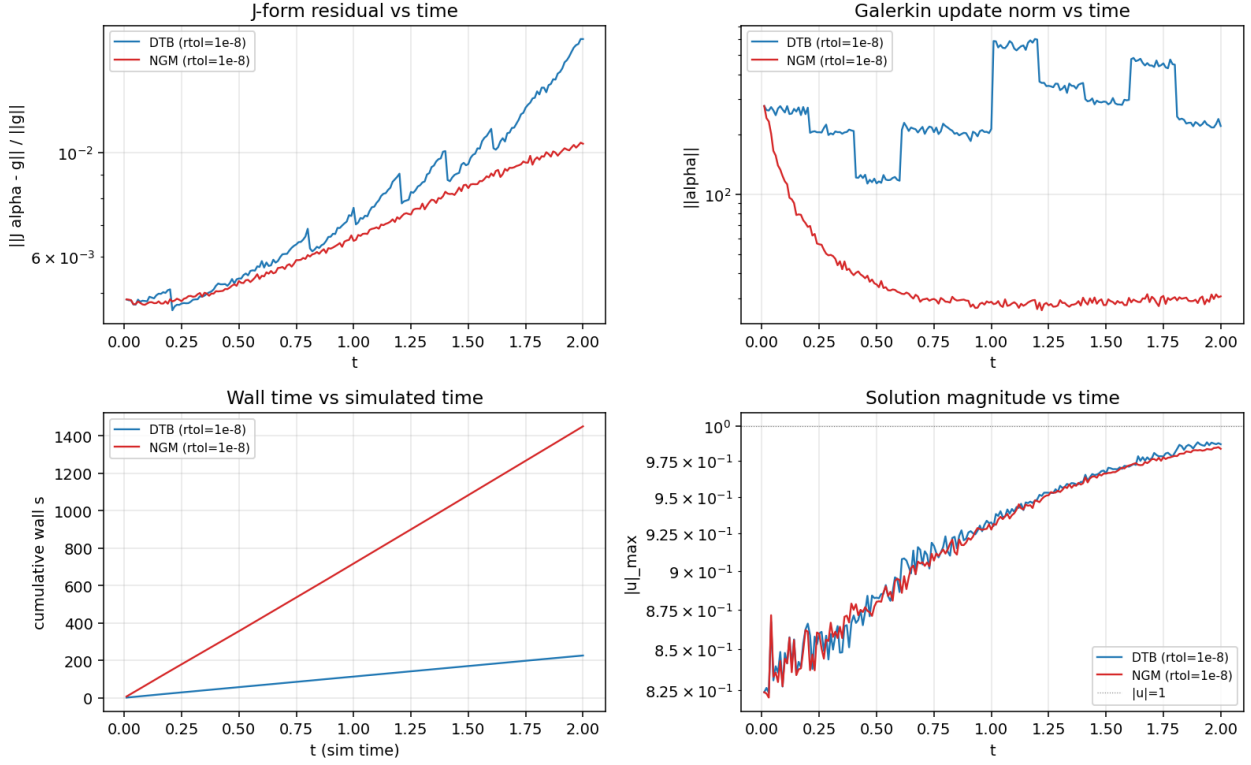


Figure 2: DTB (blue) vs NGM (red) at $\text{rtol} = 10^{-8}$. Top-left: J-form residual – DTB’s saw-tooth is the $L=20$ block-reset pattern. Top-right: $\| \alpha \|$ – NGM relaxes toward equilibrium, DTB oscillates with the block resets. Bottom-left: cumulative wall time – NGM is slower only because of the logging; the algorithmic cost is the same. Bottom-right: $|u|_{\infty}$ trajectories agree to 10^{-3} .

7 Reproduce

```

pip install -r requirements.txt

# DTB (~6 min on a 16 GB+ NVIDIA GPU)
python run_5d_ac.py --tag paper2 --fit_steps 6000 --fit_lr 3e-3
python visualize.py snapshots/run_paper2

# Side-by-side with the paper's Fig. 5 panels
python compare_to_paper.py snapshots/run_paper2

# DTB at strict truncation (~6 min)
python run_5d_ac.py --tag dtb_strict --solver lstsq --rtol 1e-8 \
    --fit_steps 6000 --fit_lr 3e-3

# NGM at strict truncation (~25 min; svdvals logging dominates)
python run_ngm.py --tag ngm_lstsq_strict --solver lstsq --rtol 1e-8

# One-step alpha vs rtol sweep (reproduces paper Table 1 trend)
python ngm_step1_failure.py --run_dir snapshots/run_paper2 --m 6000

# DTB vs NGM traces and snapshot panels
python compare_runs.py snapshots/run_dtb_strict snapshots/run_ngm_lstsq_strict
python compare_dtb_vs_ngm.py --dtb_dir snapshots/run_paper2 \
    --ngm_dir snapshots/run_ngm_lstsq_strict

```